

HUPEDCARE Assistant: A Cost-Aware, Explainable Multimodal-to-Text RAG Architecture for Pediatric Pain Education

TO-DO1¹, TO-DO2², TO-DO3³, TO-DO4⁴

¹University of MMM, Department of MMM, MMM

²University of YYY, Department of YYY, YYY

³University of ZZZ, Department of ZZZ, ZZZ

⁴University of AAA, Department of AAA, AAA

to-do1@uniovi.es, to-do2@uniovi.es, to-do3@uniovi.es, to-do4@uniovi.es

Abstract

The management of pediatric pain is a critical global healthcare priority, yet severe deficiencies persist—particularly in developing regions—due to fragmented knowledge, the underutilization of standardized scales, and a lack of accessible clinical protocols. To address these structural barriers, the transnational HUPEDCARE project establishes an open-access Learning Platform to foster academic-scientific cooperation across Europe, Africa, and Latin America. While Large Language Models (LLMs) offer a conversational interface to democratize this knowledge, their tendency to hallucinate makes their naïve deployment clinically unsafe. We present the HUPEDCARE Assistant, a cost-aware, multimodal-to-text Retrieval-Augmented Generation (RAG) architecture deployed as the platform’s intelligent engine. The system directly targets three objectives: idempotent ingestion to avoid redundant processing in heterogeneous institutional pipelines, hash-gated conditional reindexing to minimize recomputation when the corpus is unchanged, and structural explainability to preserve provenance and support source inspection. This paper formalizes the cost-aware update algorithm, details the highly decoupled software architecture, and reports an in-silico evaluation of operational efficiency and source-traceability signals, achieving an 86.7% reduction in simulated embedding calls and estimated direct embedding-token expenditure under the specified 30-cycle update schedule, together with cited-source agreement of 80% without provenance and 95% with provenance in a paired 20-question experiment. Finally, it outlines a prospective clinical protocol to evaluate real-world factual grounding and safety.

Keywords: Pediatric Pain Management, Retrieval-Augmented Generation (RAG), Explainable AI (XAI), Clinical Decision Support Systems, Software Architecture.

1 Introduction

The adequate management of pediatric pain is a critical global healthcare priority, yet it remains significantly under-addressed, particularly in developing regions. A comprehensive analysis conducted across academic and clinical institutions in Africa, Latin America, and Europe has identified severe, systemic deficiencies in children’s health. These include a fundamental

lack of awareness regarding pediatric pain, poor knowledge of pathophysiological mechanisms, the non-use of standardized pain measurement scales, and an absence of clear action protocols [? ?]. Furthermore, structural difficulties in training health personnel often lead to a reliance on traditional, non-evidence-based practices and severe gaps in both pharmacological and non-pharmacological treatments.

To address these educational and structural barriers, the HUPEDCARE (Humanisation of Pediatric Pain Care) project [?] was established as a transnational digital transformation initiative. Its primary objective is to modernize higher education and professional training by developing a specialized, open-access technological platform. This ecosystem is designed to fulfill four strategic objectives: (1) develop technological solutions for connectivity, collaborative work, and joint learning; (2) generate applied knowledge across continents; (3) establish specialized technological learning centers; and (4) dynamically disseminate new clinical content at both academic and non-academic levels.

While the HUPEDCARE Learning Platform provides the collaborative infrastructure, its operational success depends on translating complex clinical protocols into accessible, real-time guidance for teachers, students, and healthcare professionals. Large Language Models (LLMs) offer a conversational interface that may widen access to this knowledge [?]. However, the deployment of generic LLMs in clinical education is hindered by their tendency to "hallucinate" plausible but clinically unsafe content [?]. In pediatric pain management, where treatment protocols may require precision, such errors can cause harm and undermine the goals of academic-scientific cooperation.

Retrieval-Augmented Generation (RAG) can support evidence-aware responses by decoupling knowledge storage from linguistic generation and retrieving content from curated external corpora [?]. Retrieval alone does not ensure that the selected evidence is relevant, complete, or correctly applied, so it cannot by itself establish clinical safety. Nevertheless, deployment in a transnational, resource-constrained ecosystem poses software engineering challenges. Clinical knowledge within the consortium is heterogeneous, continuously updated, and subject to strict operational cost constraints. Continually rebuilding dense vector indexes over massive international corpora may be computationally and financially burdensome for participating institutions.

To fulfill the project's digital transformation mandate, we introduce the HUPEDCARE Assistant, a cost-aware multimodal-to-text RAG system deployed as the intelligent engine of the HUPEDCARE ecosystem. Sitting at the intersection of Medical Informatics and Applied Software Engineering, our work offers the following core contributions:

1. **Idempotent Ingestion Pipeline:** An orchestrator-driven workflow that synchronizes heterogeneous sources into a unified corpus and avoids redundant processing when the underlying content has not changed.
2. **Hash-Gated Conditional Reindexing:** A cost-aware indexing mechanism that uses corpus hashing to skip unnecessary recomputation when the corpus is unchanged, and triggers full re-indexing only when updates are detected.
3. **Structural Explainability (XAI):** The integration of Explainable AI principles into the data abstraction workflow. Provenance metadata is prefixed to data chunks, facilitating source inspection and auditability by healthcare professionals and academic trainers.

The scientific contribution is the formalized, reproducible integration of these mechanisms into a single architecture for transnational, resource-constrained pediatric pain education, rather than novelty in any individual component. Consistent with this scope, the paper is best read as a software-architecture contribution: it evaluates operational efficiency and evidence traceability, not clinical effectiveness or retrieval quality improvements, which remain future work (Section 7).

The rest of this paper is structured as follows: Section 2 provides a comprehensive review of related work. Section 3 details the system architecture and implementation mechanisms. Section 4 presents a comparative analysis with existing clinical knowledge support paradigms. Section 5 outlines our clinical validation and experimental protocol. Section 6 discusses the system’s limitations and data privacy considerations. Finally, Section 7 concludes the paper and outlines directions for future research.

2 Related Work

2.1 Conversational Agents and LLMs in Healthcare

Traditional Clinical Decision Support Systems (CDSS) have typically relied on expert rules, ontologies, and manually curated knowledge bases. Representative systems, including early large-scale oncology assistants, were deterministic and documentation-oriented, but also brittle, expensive to maintain, and difficult to adapt to open-ended real-world questions [? ?]. In practice, their interaction model is often poorly aligned with caregivers and non-specialist users, who may struggle to translate everyday concerns into structured query formats.

Large Language Models (LLMs) have expanded clinical AI by enabling free-text interaction and broad semantic generalization. Domain-focused systems such as Med-PaLM report strong benchmark performance in medical QA settings and near-expert accuracy on multiple-choice exams [? ?]. Recent surveys describe a rapid proliferation of LLM-based assistants for triage, education, documentation support, and guideline navigation across specialties, including pediatrics [? ?]. These systems often emphasize usability and ease of deployment, but their clinical reliability still depends strongly on corpus design, prompt policies, and organizational governance.

However, parametric knowledge alone remains insufficient for safety-sensitive use: hallucinations, stale knowledge, and weak source traceability remain central concerns, particularly for pediatric pain contexts where dosage, escalation thresholds, and contraindications must be exact [? ?]. Empirical evaluations document that even high-performing LLMs can generate plausible but unsupported recommendations, and that naive prompting provides limited guarantees in edge cases [?]. As a result, there is growing consensus that LLM-based clinical assistants should be grounded in curated knowledge sources and embedded into explicit software architectures, rather than being exposed as standalone chatbots.

These limitations motivate a hybrid paradigm in which LLMs are used as language interfaces over externally managed knowledge stores, instead of being treated as autonomous decision-makers. The unresolved gap is not only model quality, but operational maintainability: healthcare deployments need update mechanisms that are frequent, auditable, and affordable under constrained infrastructure, while providing interaction patterns that are understandable to caregivers and trainees. This motivates architectures that combine conversational interfaces with disciplined knowledge ingestion, synchronization, and monitoring, which is the space where

the HUPEDCARE Assistant is positioned.

2.2 Retrieval-Augmented Generation in Medicine

Retrieval-Augmented Generation (RAG) is now a standard approach for grounding LLM outputs in external corpora [?]. By separating retrieval from generation, RAG improves evidential control and enables corpus updates without model retraining. In healthcare, early medical RAG studies and benchmark initiatives (e.g., MedRAG) report gains in factuality and lower hallucination rates compared with purely parametric approaches, particularly on question answering tasks derived from guidelines and textbooks [? ?]. RAG has also been explored for summarizing clinical notes, supporting coding workflows, and assisting with literature review, frequently improving perceived safety relative to untethered chat-based systems.

Despite these advantages, much of the literature evaluates RAG under controlled conditions with relatively clean, text-centric datasets. Production constraints common in hospital ecosystems are less represented: heterogeneous source formats (PDF protocols, scanned forms, spreadsheets, multimedia educational material), legacy file-transfer workflows (FTP-based ingestion, manual uploads), intermittently updated SQL backends, uneven network connectivity, and strict API-cost ceilings. General-purpose orchestration frameworks (e.g., generic LLM toolkits) accelerate prototyping but often treat embedding pipelines as stateless components, with full reindexing assumed at each update cycle and limited visibility into cost behavior over time.

Few works report concrete cost models, describe how indexing and synchronization behave over months-long deployments, or quantify the effect of update strategies on maintenance burden. Likewise, the interaction between corpus-change frequency, embedding dimensionality, and provider pricing is rarely formalized, even though it determines whether a deployment is economically viable in low- and middle-income settings.

The main gap for our setting is therefore operational: a medically grounded RAG system must combine factuality with maintainable synchronization and explicit cost control. HUPEDCARE Assistant addresses this by coupling idempotent ingestion with hash-gated conditional reindexing, making these mechanisms first-class architectural elements rather than implicit framework defaults, and by articulating a minimal quantitative reporting template to characterize cost and grounding behavior.

2.3 Explainable AI and Evidence Traceability

Explainable AI (XAI) is widely recognized as essential for trustworthy AI in safety-critical domains [?]. However, many XAI methods remain post-hoc and model-centric (e.g., saliency maps, attribution scores, or local surrogate models around black-box predictors). These techniques can be informative at population level and for algorithm auditing, yet provide limited assurance for single patient-facing exchanges where clinicians and caregivers require concrete documentary evidence and clear accountability.

Healthcare practice emphasizes documentation, provenance, and auditability as core components of professional responsibility and regulatory compliance [?]. In regulated settings, clinicians are expected to justify decisions with reference to guidelines, protocols, and patient records, not only to abstract model explanations. For LLM applications, this suggests an architectural shift: instead of explaining outputs only after generation, systems should preserve

evidence lineage through ingestion, retrieval, and response construction, and expose that lineage in a form that is intelligible to clinicians and educators.

Recent work on “structural” or “procedural” XAI argues that traceability should be engineered into the data and software pipeline [?], for example by retaining source identifiers through embedding, retrieval, and answer assembly, or by constraining answer templates to state the provenance of key recommendations. This moves XAI from an optional add-on to a design principle that shapes how data structures, chunking schemes, and prompts are defined.

HUPEDCARE Assistant adopts this structural XAI perspective by injecting filename-level provenance headers into indexed content and instructing response generation to expose source references as part of the answer surface. The gap addressed here is evidence traceability: not model interpretability in isolation, but evidence paths that can be inspected in clinical and educational workflows, logged for audit, and used to challenge or correct answers when needed. This architecture facilitates source inspection; it does not demonstrate calibrated user trust or that caregivers correctly interpret cited evidence. The current filename-based labels are not immutable identifiers and can be ambiguous under renaming, reuse, or content revision; stable document identifiers, content hashes, and version metadata are needed for complete auditability. In this sense, the system preserves basic provenance in the architecture and corpus rather than relying only on the LLM.

2.4 Pediatric Pain Management and Digital Support for Caregivers

Pediatric pain is a core quality-of-care domain with long-term implications for child outcomes, family burden, and health-service utilization [?]. International and national guidelines emphasize structured pain assessment, multimodal interventions, age-appropriate communication, and caregiver education [? ?]. Integrative reviews in low- and middle-income countries describe persistent gaps in knowledge, scale usage, and protocol adherence, often linked to limited training opportunities and resource constraints [?].

In practice, these recommendations are distributed across heterogeneous artifacts (protocol documents, scales, discharge instructions, training manuals, and educational media), often in multiple languages and formats and maintained by different institutional actors. This fragmentation complicates timely access at the point of need, particularly for caregivers and trainees who lack direct access to institutional knowledge management systems or who interact with the health system through mobile devices and intermittent connectivity.

Current caregiver-facing digital support remains fragmented, frequently limited to static web content, PDF repositories, or fixed decision trees with limited personalization and little transparency about underlying evidence. Few existing tools combine multimodal guidance (e.g., textual instructions plus behavioral pain scales) with explicit provenance and clear safety guardrails. The problem is amplified in transnational and resource-constrained settings, where infrastructure heterogeneity and operational costs shape what can be sustainably deployed and maintained over time.

HUPEDCARE Assistant is positioned to address this intersection: not by creating new clinical guidance, but by operationalizing existing pediatric pain knowledge through a multimodal, provenance-aware, and cost-controlled RAG architecture suitable for real institutional workflows across continents. The system seeks to bring guideline-aligned, evidence-traceable answers closer to caregivers and trainees, while respecting institutional constraints on infrastructure, staffing,

and budget.

2.5 Operational and Regulatory Frameworks for Clinical AI

Beyond model architectures, clinical AI deployment is constrained by operational and regulatory frameworks that govern software lifecycle, information security, and risk management. Standards such as IEC 62304 [?] define processes for the development and maintenance of medical device software, emphasizing traceability of requirements, disciplined change management, and verification activities across the lifecycle. Information security frameworks (e.g., ISO 27001 [?]) similarly highlight the need for controlled access, logging, and incident response in health-related systems.

Many LLM and RAG prototypes in healthcare do not yet align with these frameworks, focusing primarily on model performance rather than on how ingestion pipelines, logging, configuration management, and provider APIs will be governed in practice. As a result, there is a gap between proof-of-concept systems and architectures that could be credibly integrated into institutions with regulatory oversight, especially when cross-border data flows and third-party AI providers are involved.

HUPEDCARE Assistant draws on this space by structuring its ingestion and inference components as separable services and defining observable interfaces (health and readiness endpoints and basic operational counters). These choices are informed by lifecycle and information-security principles, but they do not demonstrate conformity with IEC 62304 or ISO 27001. The current implementation is intended for educational and decision-support uses rather than as a certified medical device; formal risk management, verification evidence, and compliance assessment remain necessary before any regulated clinical deployment.

2.6 Synthesis of Research Gap and Objective Mapping

Across these lines of prior work, three unresolved requirements motivate this study: (G1) economically sustainable knowledge-refresh cycles under heterogeneous hospital and educational data pipelines, (G2) avoiding redundant recomputation when the corpus is unchanged, and (G3) evidence traceability that is usable in real clinical-educational interactions and compatible with emerging governance expectations. Based on the literature reviewed, we found limited evidence of systems that combine all three mechanisms in pediatric pain education under resource-constrained, transnational conditions. The contribution of HUPEDCARE Assistant lies in their formalized integration and evaluation within this applied setting, rather than in claiming novelty for individual components.

To make this research logic explicit, Table 1 maps each identified gap to its corresponding objective, architectural response, and location in the manuscript.

Table 1: Research-gap to objective coverage in HUPEDCARE Assistant.

Research gap	Objective in this article	Architectural response	Where addressed in the article
G1: Economically sustainable updates under heterogeneous pipelines	Idempotent Ingestion Pipeline	Idempotent ingestion workflow that reconciles distributed sources and avoids redundant processing	Section 3 (Layers 1–2), Section 4, and Section 5 (H1)
G2: Operational cost control when the corpus is unchanged	Hash-Gated Conditional Reindexing	Hash-based state gate that skips embedding when the corpus hash is unchanged and rebuilds only on detected updates	Section 3 (Layer 3), Section 4, and Section 5 (H2)
G3: Source-level evidence traceability in clinical-educational interactions	Structural Explainability (XAI)	Provenance-prefixed corpus chunks plus a source-oriented response policy	Section 3 (Layers 2 & 5), Section 4, and Section 5 (H3)

3 System Architecture

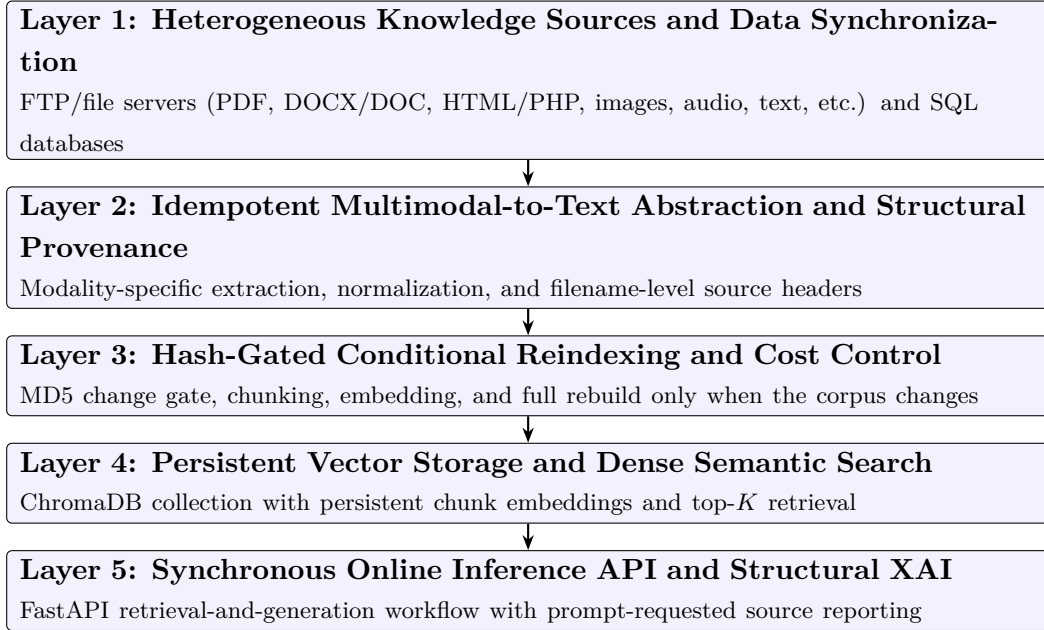


Figure 1: High-level architecture of the HUPEDCARE Assistant. The five layer titles match the corresponding architecture subsections in Section 3. File-server formats are representative rather than exhaustive.

The HUPEDCARE system follows a highly decoupled service-oriented architecture (as illustrated in Figure 1) that separates offline ETL (Extract, Transform, Load) workflows from the asynchronous, online inference API. Built on a modern Python ecosystem, the implementation explicitly adopts the Strategy and Observer design patterns to preserve high cohesion, low

coupling, and seamless extensibility across new data modalities.

3.1 Layer 1: Heterogeneous Knowledge Sources and Data Synchronization

Clinical environments require synchronization pipelines that are designed to be robust and idempotent. The system cannot rely on fragile event-driven triggers (e.g., webhook failures from legacy hospital CMS platforms); instead, it utilizes a state-reconciliation orchestrator to maintain a local, query-ready mirror of institutional knowledge across distributed file servers and relational databases.

3.1.1 FTP Synchronization Automaton

Institutional knowledge is frequently updated on distributed file servers. The `ftp_collector` module models synchronization as a state reconciliation problem between a remote file tree F_{remote} and a local metadata ledger L . In implementation, L is persisted as a lightweight JSON state file (`.metadata.json`), which preserves synchronization state across server reboots.

Let f_i be a clinical file, and let $T(f_i)$ be its last-modified timestamp. The local ledger is defined as $L = \{(f_1, T_{local}(f_1)), \dots, (f_n, T_{local}(f_n))\}$. During the synchronization epoch, the orchestrator retrieves the remote state $R = \{(f_1, T_{remote}(f_1)), \dots, (f_m, T_{remote}(f_m))\}$. The synchronization function $Sync(f_i)$ executes according to the following rules:

$$Sync(f_i) = \begin{cases} \text{Download}(f_i) \wedge \text{Update}(L, f_i) & \text{if } f_i \text{ is new or } T_{remote}(f_i) \neq T_{local}(f_i) \\ \text{Purge}(f_i) \wedge \text{Remove}(L, f_i) & \text{if } f_i \in L \wedge f_i \notin R \\ \text{No-Op} & \text{if } T_{remote}(f_i) = T_{local}(f_i) \end{cases} \quad (1)$$

The purge step removes deprecated files from the local cache and prevents stale content from being reprocessed in later stages. In the current implementation, timestamp comparison is an efficiency heuristic, not a content-integrity guarantee: a remote update that preserves its timestamp, limited timestamp resolution, copied files with preserved metadata, or interrupted downloads can evade detection or produce an incomplete local artifact. The pipeline does not yet combine modification time with remote file size and content checksums, nor does it atomically publish verified downloads. For critical clinical sources, these controls should be introduced before operational deployment.

3.1.2 Relational Database Extraction (SQL)

Clinical FAQs, feedback, and structured discharge instructions often reside in relational databases. The `sql_collector` connects to the MySQL cluster using connection parameters and queries defined in a central `config.yaml`, which keeps the extraction logic separate from the database-specific SQL statements. The resulting tuples $t_j \in \text{DB}$ are normalized into text spans and passed through a regex-based HTML sanitization pipeline. This removes structural markup (`<div>`, `<script>`, CSS classes) before downstream embedding, yielding clinical text that can be tagged with provenance information.

3.1.3 Media File Repositories (Audio and Visual Assets)

In addition to textual documents and relational records, clinical knowledge across partner institutions includes rich media assets, such as recorded clinical voice notes, audio instructions (.mp3, .wav, .m4a, .flac), and visual educational aids (.jpg, .png, .webp). These assets are synchronized via the file orchestrator into the local staging area alongside standard documents. By treating media files as first-class inputs at Layer 1, the pipeline ensures that non-textual clinical expertise enters the synchronization lifecycle and is subsequently made retrievable through Layer 2 multimodal abstraction.

3.2 Layer 2: Idempotent Multimodal-to-Text Abstraction and Structural Provenance

The diversity of medical data requires a uniform abstraction layer. The pipeline employs the Strategy design pattern to route synchronized artifacts to modality-specific extraction routines based on file type (extension-based routing in the current implementation). Let Σ denote the set of supported modalities, and let *Ext* map each modality to its corresponding text extraction routine. The resulting architecture is multimodal at ingestion but converts every supported modality into a common textual representation before embedding and retrieval; it is not an end-to-end multimodal RAG system that retrieves or reasons over the original image or audio content.

- **Binary Document Parsing:** For PDF files (.pdf), the system uses PyPDF2 to extract page text. For modern Word documents (.docx), it uses a native Python parser, and for legacy Word files (.doc) it applies a lightweight byte-level text recovery heuristic to recover readable content without requiring proprietary document software.
- **Web and Markup Content Extraction:** For web-based artifacts (.html, .htm, .php), the system uses BeautifulSoup to strip navigation bars, scripts, and styling tags, retaining only the readable article content.
- **Automatic Speech Recognition (ASR):** Clinical voice notes and recorded instructions (.mp3, .wav, .m4a, .flac) are routed to OpenAI’s *whisper-1* transcription model via API. The audio waveform is transcribed into a textual sequence $w = (w_1, w_2, \dots, w_k)$, allowing speech content to enter the same retrieval pipeline as text.
- **Vision-Language Model (VLM):** Visual clinical tools, such as the FLACC behavioral pain scale or complex flowcharts (.jpg, .jpeg, .png, .webp), are converted into semantic descriptions by the *gpt-4o-mini-2024-07-18* vision-language model via API, making their content retrievable in the downstream corpus.
- **Plain Text Passthrough:** Files already in plain-text form (.txt) are read directly with UTF-8 decoding (ignoring malformed bytes) and require no further extraction step.

All abstracted outputs are concatenated into a master corpus, $\mathcal{C}$. At this stage, each extracted block c_i is prefixed with a structural provenance header (e.g., `-- SOURCE: post_op_protocol.pdf --`). This keeps a filename-level link to the raw artifact in the local mirror attached to the derived text as it moves into vectorization and retrieval (as illustrated in Figure 2). However,

the current corpus does not store structured extraction metadata such as the ASR/VLM model version, extraction time, or a content checksum alongside each derived text block. These fields, together with stable references to the original assets, are required for a complete multimodal audit trail and are planned for future work.

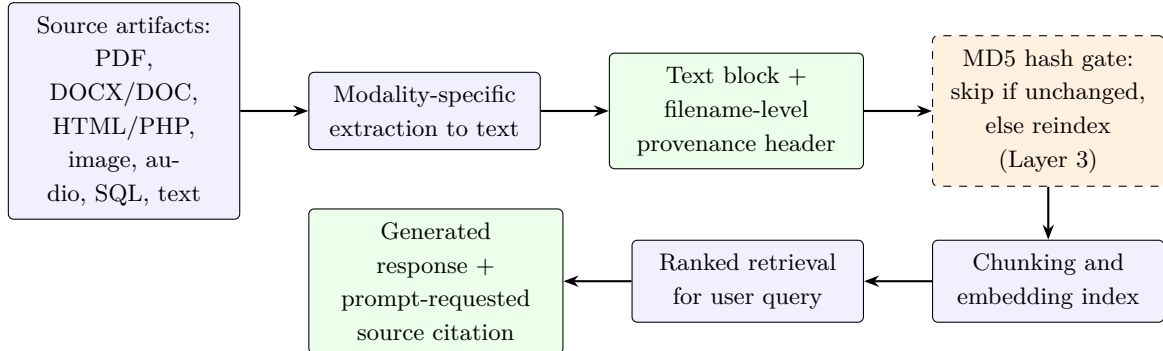


Figure 2: Conceptual evidence path in HUPEDCARE Assistant. Provenance is attached to derived textual blocks before indexing; the hash gate then determines whether embedding actually runs (Layer 3) or is skipped as a no-op. Provenance remains available in retrieved context; visible source citation in the generated response is requested by prompt and is not a deterministic guarantee.

3.3 Layer 3: Hash-Gated Conditional Reindexing and Cost Control

The transition from a raw textual corpus $\mathcal{C}$ to a high-dimensional vector space is the most computationally and financially expensive operation in the RAG lifecycle. We use ChromaDB as a local vector engine. Let N be the total token count of $\mathcal{C}$, M the number of chunks produced by the splitter, and d the embedding dimensionality ($d = 1536$, the native output dimension of OpenAI’s `text-embedding-3-small`, the model used throughout this implementation). A stateless pipeline that always rebuilds the index incurs a direct embedding-token cost proportional to $O(N)$ on every execution (since provider pricing is per token, independent of d), plus a storage and vector-insertion complexity of $O(M \cdot d)$ from materializing and persisting M dense chunk vectors of dimension d ; both are wasted whenever no content has changed.

To reduce unnecessary embedding calls, HUPEDCARE Assistant applies a hash-based state gate using an MD5 checksum of the current corpus. MD5 is used here strictly as a low-cost change-detection fingerprint and cache-gating mechanism; it is not used, and should not be interpreted, as an authenticity, integrity-protection, or security-verification control. A collision-resistant function such as SHA-256 would be preferable if the hash were ever repurposed for document-integrity auditing. Algorithm 1 summarizes the implemented update logic.

Algorithm 1 Hash-Gated Vector Reindexing in ChromaDB

Require: Previous Hash H_{prev} , Current Corpus $\mathcal{C}_{current}$, VectorDB V

Ensure: Boolean success status and updated VectorDB state

```
1:  $H_{curr} \leftarrow \text{MD5}(\mathcal{C}_{current})$ 
2: if  $H_{curr} == H_{prev}$  then
3:   return Success (No-Op)
4: end if
5:  $V.\text{clear\_collection}()$ 
6:  $Chunks \leftarrow \text{RecursiveCharacterSplit}(\mathcal{C}_{current}, \text{size}, \text{overlap})$ 
7:  $V.\text{insert}(Chunks, \text{API\_Embed}(Chunks, \text{dims} = 1536))$ 
8:  $\text{SaveMetadata}(H_{curr})$ 
9: return Success (Full Reindexed)
```

This state-gated approach skips embedding when the corpus is unchanged. When a change is detected, the current implementation performs a full collection rebuild (delete and recreate), with source-aware chunking separators that prioritize provenance headers during splitting. Consequently, the demonstrated efficiency applies to no-change cycles; a small modification in a large corpus still triggers corpus-wide chunking and re-embedding. This trade-off favors a simple, consistent index state in the present prototype, but it does not provide incremental-update efficiency.

Formally, $H(\cdot)$ denotes the MD5 hash function applied to the serialized corpus, and $\mathcal{C}_t$ denotes the corpus snapshot at synchronization step t . Here, $|\mathcal{C}_t|$ denotes corpus size (e.g., token count).

Let c_{embed} denote the marginal embedding cost per token. The practical cost model over time is:

$$\text{Cost}(t) = \begin{cases} 0 & \text{if } H(\mathcal{C}_t) = H(\mathcal{C}_{t-1}) \\ |\mathcal{C}_t| \cdot c_{\text{embed}} & \text{if } H(\mathcal{C}_t) \neq H(\mathcal{C}_{t-1}) \end{cases} \quad (2)$$

This mechanism avoids redundant re-embedding during unchanged cycles, while changed cycles still incur full reindexing cost regardless of the size of the modified source.

To distinguish monetary cost from runtime complexity, let $T(t)$ denote execution time per synchronization cycle, let M denote the number of chunks produced by the splitter, and let $T_{\text{embed}}(\cdot)$ denote the (network- and provider-dependent) wall-clock latency of the embedding API call. A practical model is:

$$T(t) \approx \begin{cases} O(|\mathcal{C}_t|) & \text{if } H(\mathcal{C}_t) = H(\mathcal{C}_{t-1}) \\ O(|\mathcal{C}_t|) + T_{\text{embed}}(|\mathcal{C}_t|) + O(M \cdot d) & \text{if } H(\mathcal{C}_t) \neq H(\mathcal{C}_{t-1}) \end{cases} \quad (3)$$

The no-change path is dominated by corpus hashing and metadata comparison. The change path is not adequately summarized by a single asymptotic term in $|\mathcal{C}_t|$ and d : it comprises (i) an $O(|\mathcal{C}_t|)$ corpus-scanning and chunking pass, (ii) the embedding-provider round trip $T_{\text{embed}}(\cdot)$, whose wall-clock duration is governed by network latency and provider throughput rather than local computation, and (iii) an $O(M \cdot d)$ cost for materializing and inserting the resulting chunk vectors into ChromaDB. In practice, $T_{\text{embed}}(\cdot)$ typically dominates the change-cycle runtime and is outside the system’s direct control.

3.4 Layer 4: Persistent Vector Storage and Dense Semantic Search

Persistent vector storage bridges offline ETL indexing with online inference. The system maintains a local ChromaDB instance storing document chunk embeddings in a 1536-dimensional vector space ($V \in \mathbb{R}^{1536}$), matching the native output dimension of the `text-embedding-3-small` model.

During offline indexing (Layer 3), chunks generated from the master corpus $\mathcal{C}$ are stored alongside their structural provenance headers. During online query processing, the user’s natural language query Q is converted into a query vector $V_q \in \mathbb{R}^{1536}$ using the same embedding model. Dimensional alignment is strictly enforced so that chunk embeddings and query vectors inhabit the exact same metric space. The system queries ChromaDB to retrieve the top- K chunks ranked by distance. The current implementation uses rank order; future work will calibrate explicit distance thresholds to filter out low-confidence context in safety-sensitive clinical inquiries.

3.5 Layer 5: Synchronous Online Inference API and Structural XAI

The online serving layer is deployed as an asynchronous FastAPI microservice. FastAPI was selected for its native integration with Starlette (providing asynchronous event loops) and Pydantic (ensuring strict request/response data validation). When a caregiver submits a query Q to the `/ask` endpoint, the payload is schema-validated via Pydantic models before executing the structured retrieval-and-generation workflow:

1. **Payload Validation and Observability:** Request payloads are strictly validated against Pydantic schemas (e.g., `Query(BaseModel)`) to reject malformed inputs before invoking downstream AI services. Operational readiness and health endpoints (`/ready` and `/health`) allow monitoring dashboards to verify connectivity to the vector collection.

2. **Safety Guardrails and Constitutional Constraints:** The current implementation applies safety guidance primarily through prompt policy: the system message instructs retrieval-grounded behavior, source reporting, and explicit abstention when evidence is missing. In addition, request payloads are schema-validated via Pydantic and the API includes operational readiness and observability endpoints. However, a dedicated pre-generation domain classifier with a deterministic hard-stop branch is not currently implemented in the request path.

3. **Structural Provenance Rendering:** Because Layer 2 injects provenance headers (`-- SOURCE: X --`) into the indexed corpus, retrieved context carries explicit source references during generation. The system prompt requests that the final answer briefly mention the source used, which was associated with improved cited-source agreement in the reported evaluation. This shifts the paradigm from post-hoc explainability to *Structural Explainability* [?]. The evidence path is inspectable in retrieved context, while source rendering in the final answer remains dependent on LLM compliance with prompt instructions.

4 Comparative Analysis

To position HUPEDCARE Assistant within the broader landscape of clinical knowledge support systems, Table 2 provides a qualitative design-level contrast with two broad paradigms: legacy rule-based CDSS (e.g., early expert systems) and generic RAG frameworks commonly used to prototype LLM applications (e.g., naive pipeline stacks). Table 3 then summarizes concrete repository-level parameters.

We emphasize that this comparison is purely architectural and conceptual, informed by published literature and code structure analysis; it is not a direct experimental benchmark against deployed third-party systems. An empirical head-to-head evaluation against legacy CDSS or commercial platforms was not conducted due to the proprietary nature of clinical CDSS software, the absence of standardized benchmark datasets for multimodal pediatric pain protocols, and fundamental differences in system objectives (rule execution vs. grounded language generation). Instead, this analysis serves to taxonomize design choices. Implementing a standardized, open-source generic RAG baseline for direct empirical comparison represents an important direction for future software-engineering work.

We deliberately scope the reported evaluation (Section 5) around *operational efficiency* (synchronization correctness, embedding-cost control) and *evidence traceability* (citation compliance), rather than *retrieval effectiveness* (e.g., recall@ k , ranking quality) or *answer correctness* against a gold clinical standard. This is a scope choice, not an oversight: H1 and H2 are software-engineering properties that hold independently of retrieval quality, since idempotent synchronization and hash-gated reindexing behave identically regardless of how well the retriever ranks results, whereas H3 concerns whether provenance is surfaced, not whether the retrieved evidence is optimal. Improving retrieval effectiveness and answer correctness against clinician-validated references are acknowledged as necessary complementary evaluations and are left for the prospective clinical protocol in Section 5.6.

The tables operationalize the three gaps defined in Section 2: G1 (knowledge updates), G2 (cost control under unchanged corpora), and G3 (source-level traceability). Safety-oriented design is included only as a cross-cutting quality attribute, not as a claim of demonstrated clinical safety. They illustrate how the integrated architecture responds to these requirements without claiming novelty for individual underlying libraries.

Table 2: Qualitative design-level comparison of clinical knowledge support paradigms. No common benchmark or shared protocol was used; this is not an experimental evaluation against implemented external systems.

Feature	Legacy CDSS	Generic RAG	HUPEDCARE Assistant
Knowledge updates	Manual / rigid	Manual / static	Automated, state-based
Operational cost	High upfront, low variable	Variable, often uncontrolled	Cost-bounded via hashing
Traceability / XAI	Rule logs, limited for users	Post-hoc explanations	Built-in provenance
Safety-oriented design (cross-cutting attribute)	Deterministic rules	Probabilistic generation	Prompt-based guardrails + grounded RAG

Interpretation note: Table 2 summarizes typical design tendencies, not universal properties of every CDSS or RAG implementation. No external system was implemented or measured under a common protocol. The first three rows map directly to G1–G3, while *safety-oriented design* is reported as a transversal quality attribute, not as evidence of validated clinical safety, and therefore is not counted as an additional research gap.

Legacy CDSS typically require manual rule authoring and extensive expert maintenance, which can slow guideline updates and limit scalability. Generic RAG frameworks simplify development but may treat embedding and retrieval as stateless components, leading to redundant computation, opaque provenance handling, and limited control over operational costs. HUPEDCARE Assistant is explicitly designed around stateful synchronization, cost-aware indexing, and structural provenance, aiming to retain documentation and auditability practices while leveraging the flexibility of RAG-based language interfaces.

4.1 Objective Implementation Profile

Table 3 summarizes the repository-level configuration and runtime mechanisms that instantiate the three objectives introduced in Section 1.

Table 3: Repository-level implementation profile of the HUPEDCARE Assistant.

Objective / mechanism	Concrete repository fact	Comparative implication
Idempotent ingestion	FTP synchronization plus SQL extraction feed a single master corpus; orphaned cache files are purged; repeated extraction is limited to new or modified files; unsupported file types are skipped by the registry	Designed to provide greater operational stability than stateless ingestion pipelines that reprocess the entire corpus on every run
Hash-gated conditional reindexing	The collector computes an MD5 hash of <code>master_context.txt</code> ; if the hash is unchanged, embedding is skipped; if it changes, the vector database is fully rebuilt; the vector layer deletes and recreates <code>rag_context</code> before adding new chunks	Designed to provide more predictable cost control than always-on re-embedding workflows, since unchanged cycles do not consume embedding API budget
Structural explainability (XAI)	Source headers of the form <code>-- SOURCE: ... --</code> are inserted into the corpus; the system prompt requests source reporting; retrieved context carries provenance into the generation step	Supports source inspection more directly than RAG stacks that pass retrieved text without explicit source labels
Serving stack	The API uses FastAPI with Pydantic validation, CORS middleware, a persistent ChromaDB collection named <code>rag_context</code> , and an OpenAI-compatible client configured by <code>base_url</code> and <code>MODEL_API_KEY</code> ; readiness and health endpoints are exposed at <code>/ready</code> and <code>/health</code>	Produces a lightweight but auditable inference layer suitable for bounded clinical-educational deployments
Retrieval and chunking parameters	The configuration uses <code>text-embedding-3-small</code> , top- $K = 6$, chunk size = 1100, and overlap = 200; the chunk splitter prioritizes <code>\n--SOURCE: </code> and <code>\n[DB </code> as separators	These values make the retrieval stage explicit and reproducible, which is preferable to hidden default settings in generic frameworks
Operational observability	Middleware tracks total requests, ask latency, successes, and errors; logs rotate with a 5 MB cap and 3 backups; the API records basic operational counters in memory	Easier to audit than opaque prototypes because runtime status and failure modes are surfaced directly in the application layer
Supported modalities	The registry covers <code>.html</code> , <code>.htm</code> , <code>.php</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.webp</code> , <code>.mp3</code> , <code>.wav</code> , <code>.m4a</code> , <code>.flac</code> , <code>.docx</code> , <code>.doc</code> , <code>.pdf</code> , and <code>.txt</code>	More explicit than many generic RAG stacks because modality routing is declared in code rather than inferred at runtime
Configuration anchors	The prompt policy enforces retrieval-first behavior; the transcription model is <code>whisper-1</code> ; the vision model and chat model are pinned to <code>gpt-4o-mini-2024-07-18</code> ; the server allows the specified production origins and a local development regex	These fixed settings make the system reproducible and reduce ambiguity in deployment and evaluation

The resulting implementation profile is intentionally specific: ingestion is incremental at the file level, vector storage is persistent but rebuilt when the corpus hash changes, and provenance is retained as a first-class artifact rather than reconstructed after generation. This makes the architecture suitable for a software-engineering comparison with CDSS and generic RAG systems on concrete dimensions such as update discipline, operational cost, and traceability.

4.2 Mechanism-Level Comparison

Table 4 condenses the practical differences that matter for deployment and evaluation. It is intentionally narrower than Table 2: instead of comparing general system families, it compares the specific mechanism choices that determine update behavior, provenance handling, and runtime visibility.

Table 4: Mechanism-level comparison of the three architecture families. Conceptual comparison only; no common benchmark or shared protocol was applied.

Dimension		Legacy CDSS	Generic RAG	HUPEDCARE Assistant
Update mechanism		Manual rule maintenance or periodic curated releases	Often stateless re-embedding or framework-managed refreshes	Idempotent ingestion plus hash-gated conditional reindexing
Evidence path		Rule logs and documentation artifacts	Retrieved text plus optional post-hoc explanation	Source-prefixed chunks plus prompt-requested source reporting
Input diversity		Usually structured or manually normalized inputs	Mostly text-first retrieval pipelines	Extension-based routing for <code>.pdf</code> , <code>.docx</code> , <code>.doc</code> , <code>.html</code> , <code>.php</code> , image, audio, text, and SQL-derived content
Runtime visibility		Often externalized to platform tooling	Varies by stack and is frequently implicit	Built-in <code>/health</code> , <code>/ready</code> , in-memory counters, and rotating logs
Cost control		Dominated by expert maintenance effort	Often shifts cost to repeated embedding and retrieval calls	Hash gate prevents embedding on unchanged corpora
Primary observable consequence		Change requires manual intervention and expert review	Cost and provenance behavior depend on framework defaults	Unchanged cycles generate no embedding work, while retrieved answers retain explicit source lineage

Taken together, Tables 2, 3, and 4 separate three layers of evidence: conceptual positioning, repository-level implementation facts, and the concrete mechanisms that determine whether the objectives are measurable in practice.

5 Architectural Validation and Prospective Clinical Evaluation Protocol

All *in-silico* simulations and offline evaluations reported in Phase 1 were executed on a VPS instance (1 vCPU core, 4 GB RAM, 50 GB NVMe disk) running Ubuntu 24.04 LTS. This reports the execution environment for the completed architectural tests; it does not establish end-to-end throughput, latency, or operational viability under real deployment conditions.

To inform eventual clinical deployment, the HUPEDCARE Assistant incorporates selected lifecycle and information-security practices described by established industrial standards. The repository includes documented configuration, source-controlled code, basic operational logging, and focused tests for the reported architectural mechanisms. These measures do not establish compliance with ISO 27001 or IEC 62304, nor do they substitute for the documented risk-management, verification, validation, and quality-system activities required for clinical acceptance.

To separate completed software-engineering evidence from future clinical evidence, the evaluation strategy is divided into two distinct phases:

- **Phase 1: Architectural Validation (Completed).** A series of controlled *in-silico* experiments and deterministic simulations designed to verify system idempotency, operational cost-efficiency, and structural traceability. The results of this phase are reported in this manuscript to validate our software engineering claims.
- **Phase 2: Clinical Validation (Prospective Protocol).** A planned empirical evaluation involving medical experts to assess real-world factual accuracy, safety, and clinical acceptance. This phase is presented here as a formal experimental protocol for future execution.

Accordingly, the hypotheses defined below (H1–H3b) and their corresponding outcomes reflect completed computational benchmarks, whereas the clinical evaluation procedures function as the reporting framework for upcoming field deployments.

5.1 Hypotheses

- **H1 (Idempotent Ingestion):** The idempotent ingestion pipeline will preserve a stable local mirror of heterogeneous institutional sources while avoiding redundant processing of unchanged files.
- **H2 (Hash-Gated Conditional Reindexing):** Under the simulated 30-cycle update schedule, hash-gated conditional reindexing will reduce estimated direct embedding-token expenditure by at least 85%, compared to a stateless full-reindex baseline that re-embeds the corpus on every run.
- **H3a (Source Traceability):** Structural provenance injection is associated with improved agreement between cited sources and retrieved context relative to a baseline without provenance headers.
- **H3b (Faithfulness):** Provenance does not substantially degrade retrieval-grounded faithfulness. This is treated as a descriptive, exploratory comparison in the present study; a formal non-inferiority threshold is not tested.

5.2 Gap Closure Matrix and Remaining Work

Table 5 states, in alignment with H1–H3, how each gap is addressed in the current article and what work still remains to fully close it.

Table 5: Gap closure status, evidence, and pending work.

Gap	Addressed by (section and mechanism)	Current evidence in this manuscript	Remaining work
G1: Sustain-able updates	Section 3 (Layers 1–3): idempotent ingestion + hash-gated conditional reindexing	Validated by controlled idempotency test: 100 % skip rate on unchanged corpus, selective re-extraction on timestamp change, orphan purge confirmed (H1 PASS); full results in Section 5.3	Real-deployment FTP and SQL measurement, plus file-size and checksum verification and atomic download publication, pending
G2: Cost control under unchanged corpora	Section 3 (Layer 3): hash-gated conditional reindexing; Section 4 (comparison and mechanism-level analysis); Section 5 (H2)	Deterministic simulation: 86.7 % reduction in embedding calls and estimated direct embedding-token cost over the specified 30-cycle schedule (H2 threshold met under this simulated schedule); full results in Section 5.4	Live API measurement; end-to-end cost accounting; and sensitivity analysis across corpus sizes, update rates, and update granularities pending
G3: Source-level evidence traceability	Section 3 (Layers 2 & 5): provenance headers + source-oriented response policy; Section 4 (comparison and implementation profile); Section 5 (H3)	Quantified by a paired 20-question experiment: faithfulness scores 0.948 (baseline) and 0.934 (HUPEDCARE Assistant); cited-source agreement with the paired original retrieval context 80.0 % (baseline) → 95.0 % (HUPEDCARE Assistant); full results in Section 5.5	Confidence intervals, repeated runs, independent and human evaluation, citation-entailment auditing, and adversarial or unanswerable prompts pending

5.3 H1: Idempotent Ingestion—Controlled Test

To validate H1, we executed a controlled idempotency test (`scripts/test_h1_idempotency.py`) directly exercising the mtime-based cache guard from the collector pipeline, using 12 synthetic clinical source files in two supported text-oriented formats (`.txt` and `.html`). The test runs three passes over a temporary directory that mirrors the `TEMP_DOWNLOADS` and `CACHE_TEXT` folder structure used in production:

- **Pass 1 (fresh run):** No cache exists. All 12 files are extracted and written to the cache. This establishes the initial mirror.

- **Pass 2 (unchanged corpus):** Source files are unmodified. The mtime guard correctly identifies all 12 cache entries as current and skips every extraction. Skip rate: **100 %**; re-extraction count: **0**.
- **Pass 3 (partial update):** 3 source files are modified (content appended, mtime advanced) and 1 source file is deleted. The pipeline re-extracts exactly the 3 modified files, skips the 8 unchanged ones, and purges the 1 orphaned cache entry. Skip rate among the 11 files still present: **72.7 %**.

All three behavioral invariants required by H1 pass in the controlled test: the unchanged corpus yields zero redundant extractions, selective re-extraction occurs for the modified fixtures, and orphan purge removes the stale cache entry. These results support the current timestamp-based idempotency claim for the tested conditions (**PASS**). They do not establish robust change detection under timestamp-preserving updates, limited server timestamp resolution, failed transfers, or reused filenames; these cases require file-size and checksum verification with atomic download publication.

5.4 H2: Embedding Cost Efficiency—Simulation

To produce a controlled, reproducible quantification of H2 prior to full deployment, we executed a deterministic cost simulation (`scripts/test_h2_cost_simulation.py`) replicating 30 consecutive update cycles. The simulation uses the actual HUPEDCARE Assistant corpus as the initial state (65 852 bytes; approximately 16 500 tokens estimated as file size divided by four characters per token), applies four controlled corpus changes at cycles 3, 8, 15, and 22 (update rate 13.3 %), and evaluates both the stateless baseline and the hash-gated strategy in parallel for each cycle. No real embedding API calls are issued; token counts are heuristic estimates and costs are derived from the configured rate of \$0.020 per million tokens for `text-embedding-3-small`.

Under this protocol, the baseline incurs 30 embedding calls and processes 496 702 estimated tokens at an estimated direct embedding-token cost of \$0.0099 over the 30-cycle simulation. The hash-gated strategy triggers only 4 calls (one per detected corpus change), processing 66 222 estimated tokens at a cost of \$0.0013. This corresponds to an **86.7 % reduction** in embedding-call count and estimated direct embedding-token expenditure, meeting the 85 % threshold stated in H2 under this specific 30-cycle simulation. This reduction follows directly from the simulated update schedule rather than being a universal system property: for T total synchronization cycles and k cycles containing a detected corpus change, the hash-gated strategy triggers embedding on exactly k cycles, giving a reduction ratio $1 - k/T$. In this experiment, $T = 30$ and $k = 4$, so $1 - 4/30 = 86.7 %$; a different update schedule (e.g., a higher change rate k/T) would yield a proportionally smaller reduction.

Because $1 - k/T$ is a closed-form, deterministic function of the update schedule alone (independent of corpus size, tokenizer, or provider pricing), its behavior at other change rates can be reported analytically without additional simulation runs. Table 6 extends the single 13.3 % data point reported above across a range of change rates k/T , making explicit that the reduction is a property of the chosen schedule rather than a fixed system guarantee; sensitivity to corpus size and update granularity (e.g., partial vs. full-document changes) still requires the live-provider measurements identified as future work below.

Table 6: Analytical sensitivity of the embedding-cost reduction ratio $1 - k/T$ to the update-cycle change rate k/T , holding $T = 30$ cycles fixed. Values are computed directly from the closed-form formula, not from additional simulation runs.

Changed cycles k (of $T = 30$)	Change rate (k/T)	Reduction ($1 - k/T$)
1	3.3 %	96.7 %
2	6.7 %	93.3 %
4 (this experiment)	13.3 %	86.7 %
8	26.7 %	73.3 %
15	50.0 %	50.0 %
23	76.7 %	23.3 %

As the table shows, the reported 86.7 % figure sits close to the boundary of the 85 % threshold stated in H2: the reduction ratio drops below 85 % once the change rate exceeds roughly 15 % (i.e., $k \geq 5$ of $T = 30$ cycles), only marginally above the 13.3 % rate used in this experiment. This dependency is intrinsic to any hash-gated (all-or-nothing) reindexing strategy and motivates the chunk-level incremental hashing discussed as future work (Section 7), which would decouple the reduction ratio from whole-corpus change frequency.

While the absolute dollar expenditure in this initial pilot simulation appears modest due to the compact corpus size ($\sim 16\,500$ estimated tokens), the key engineering insight is that embedding work scales with corpus size while vector materialization additionally scales with the number of chunks and embedding dimension. In production deployments across the 15 partner universities of the transnational HUPEDCARE consortium, clinical corpora are projected to scale to millions of tokens ($10^6 - 10^8$ tokens) subject to frequent automated synchronization checks (e.g., hourly or daily CI/CD cron jobs). Under a stateless reindexing paradigm, unconstrained daily re-embedding across distributed nodes creates substantial cumulative financial and computational overhead. In this synthetic simulation, the unchanged path requires only a local hash calculation; its measured runtime is not a production latency guarantee. The hash-gated state gate therefore makes embedding budget depend on detected content revisions rather than execution frequency, subject to the update schedule and the costs of hashing and downstream rebuilds.

The simulation is fully reproducible and parameterizable via `--cycles`, `--changes`, and `--corpus` arguments. Future evaluation should include sensitivity analyses across corpus sizes (e.g., 10^4 , 10^5 , and 10^6 documents), change rates, and small versus large updates, together with live-provider measurements and end-to-end cost accounting.

5.5 H3: Structural Explainability—LLM-as-Judge Evaluation

To evaluate H3, we ran a controlled experiment (`scripts/test_h3_ragas.py`) using 20 clinically grounded questions designed to be answerable from the HUPEDCARE Assistant corpus. The same questions were submitted to two conditions: `WITHPROVENANCE`, which uses the production corpus with `--SOURCE: ...--` headers, and `NOPROVENANCE`, which uses the same corpus with all source headers stripped. Both conditions share the same retrieval configuration (`top-K = 6`, `text-embedding-3-small`), system prompt, and generation model (`gpt-4o-mini-2024-07-18`).

Faithfulness was computed with a LLM-as-judge protocol that reproduces the RAGAS faithfulness algorithm without its dependency stack: (1) the judge extracts individual factual claims from each answer; (2) for each claim, it determines whether the claim is supported by the retrieved context chunks; (3) faithfulness is the ratio of supported claims, averaged over the 20 samples. Citation evaluation has two separate deterministic stages: *citation presence*, which detects source-like labels with a regular expression, and *cited-source agreement*, which checks whether the cited filename appears in a retrieved provenance header or whether a database citation is used with a retrieved database record. The latter is still a source-presence check, not citation entailment: it does not establish that the cited source supports every claim in the answer.

Both conditions achieve high faithfulness (WITHPROVENANCE: 0.934; NOPROVENANCE: 0.948), with a minor observed difference of 0.014 that slightly favors the no-provenance condition. This gap is within the expected run-to-run variability of LLM-as-judge faithfulness scoring at this sample size and should not be interpreted as evidence that provenance headers systematically reduce grounding quality; a repeated-measures design with multiple runs would be required to characterize this variability directly. The paired cited-source agreement audit yields 80.0 % (16/20) for NOPROVENANCE and 95.0 % (19/20) for WITHPROVENANCE. For the baseline, each cited filename or database label is checked against the contexts retrieved for the same question in the provenance condition. This is still a source-association check, not entailment: it does not establish that the source supports every claim in the answer. Provenance is associated with better source agreement in this audit, while faithfulness remains high in both conditions; the small sample does not support causal or generalizable conclusions.

The evaluation supports H3a: paired cited-source agreement is higher with provenance (95.0%) than without provenance (80.0%). H3b remains descriptive rather than formally confirmed: faithfulness is high in both conditions, but the observed score is 0.934 with provenance versus 0.948 without provenance (delta -0.014), and no non-inferiority threshold or confidence interval was prespecified. This $n = 20$ evaluation is therefore an initial feasibility check rather than a statistically powered generalizable trial. These results must not be interpreted as establishing strong causality or deterministic guarantees. Provenance injection provides structural context that may encourage source agreement, but generation remains fundamentally probabilistic and model-dependent. Limitations include the small evaluation sample, the absence of confidence intervals, variance across repeated runs, and potential shared biases from using an LLM as judge. The current audit measures source association only; it does not establish citation entailment, precision, or completeness. Future work requires expanding to larger, statistically powered query suites, auditing entailment/precision/completeness alongside presence, and including unanswerable and adversarial queries with expert human validation.

5.6 Clinical Evaluation Protocol

A panel of licensed pediatricians (target: at least 5 independent raters, drawn from the HUPED-CARE consortium’s partner institutions) will evaluate a pilot set of at least 60 simulated caregiver queries derived from real clinical interactions, sampled to cover the corpus’s major topical categories (pharmacological dosing, non-pharmacological interventions, pain-scale usage, and contraindications) in proportions matching the underlying corpus. Each query-response pair will be independently scored by at least 3 raters. The final sample size will be determined

before data collection using a prespecified precision analysis for the target Fleiss’ Kappa confidence interval, rather than fixed solely by recruitment convenience. Outputs will be rated on a 5-point Likert scale measuring *Factual Accuracy* (correctness against the cited source), *Safety* (operationally defined as the absence of dosage, contraindication, or emergency-escalation errors capable of causing patient harm), and *Source Traceability* (whether the cited source verifiably supports the claim). Responses scored below a prespecified safety threshold (mean rating ≤ 2 of 5 on the *Safety* item from any rater) will be flagged for mandatory clinical-panel adjudication. Disagreements between raters exceeding one Likert point will be resolved through a structured consensus discussion; unresolved disagreements will be adjudicated by a designated senior clinician not involved in the initial scoring.

Inter-rater reliability (IRR) across the medical panel will be rigorously computed using Fleiss’ Kappa (κ) [?]:

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e} \quad (4)$$

where $\bar{P}$ is the observed degree of agreement and $\bar{P}_e$ is the probability of random agreement. Finally, source-citation compliance will be measured using a compact audit rubric that checks whether the answer explicitly references the retrieved provenance headers.

6 Discussion and Limitations

The current implementation is oriented to institutional knowledge ingestion (FTP files and configured SQL queries) and does not include direct connectors to hospital EHR systems. In practice, privacy risk is reduced when deployed on guideline-centered and de-identified corpora; however, this is an operational governance property, not a hard technical guarantee. Because SQL extraction is configurable, privacy and regulatory compliance depend on data-source curation, de-identification procedures, and institutional access-control policies.

Interactions with external models are performed through provider APIs configured via `base_url` and API keys, and the default configuration uses HTTPS endpoints. The codebase also includes rotating application logs and basic request-level observability metrics for the API and ingestion services. Nevertheless, training-data retention policies are determined by the selected provider and contractual terms, and are not enforced by this repository. Likewise, the current logs are operational rather than full clinical audit trails.

From a governance perspective, HUPEDCARE Assistant is intended as an educational and decision-support assistant, not an autonomous diagnostic system. The prompt policy requests retrieval-first behavior (answer from retrieved context, report source, and avoid guessing), but it is not a deterministic clinical safety control or a formal regulatory certification. The current implementation has no domain classifier, minimum retrieval-similarity threshold, detector for dosage, emergency, or contraindication queries, deterministic refusal rule, or programmatic verification that a cited source supports a high-risk claim. It also does not collect patient-specific factors such as age, weight, allergies, or clinical context before responding. Consequently, the current system must not be used to resolve urgent, diagnostic, dosing, contraindication, or treatment-escalation questions; these require direct clinical assessment. In the same spirit, model reproducibility is strengthened by pinning explicit model identifiers in configuration (e.g., `gpt-4o-mini-2024-07-18` and `text-embedding-3-small`) instead of relying on unversioned defaults.

The architecture also improves technological portability. The same inference code can target different OpenAI-compatible providers by changing configuration, which reduces coupling to a single vendor and supports long-term sustainability in resource-constrained institutions. However, claims about equivalent quality or latency on heavily quantized small open-weight models remain a hypothesis until validated with dedicated benchmarks.

Beyond regulatory and operational considerations, this project supports the view that reducing hallucinations in healthcare assistants is largely a software-engineering problem: robust synchronization, explicit provenance, retrieval-grounded prompting, and observable runtime behavior. At the same time, important limitations remain. The current global-hash strategy rebuilds the complete vector collection when any corpus content changes, so its cost advantage is limited to unchanged cycles and may diminish for large, frequently updated corpora. A central additional limitation is the "lost in the middle" effect, where long retrieved contexts can dilute relevant evidence [?].

7 Conclusion and Future Work

HUPEDCARE Assistant demonstrates a reproducible architecture that integrates state-reconciled ingestion, hash-gated indexing, and structural provenance for resource-constrained pediatric pain education; the current evidence supports operational feasibility, not clinical safety. Consistent with the software-architecture framing adopted throughout, this paper deliberately reports operational efficiency and evidence-traceability results rather than clinical effectiveness claims: retrieval-grounded prompting and runtime observability are insufficient safeguards for safety-sensitive clinical use without deterministic controls and human evaluation.

Building upon the HUPEDCARE Assistant architecture, future research will focus on four directions. First, we will replace global-hash rebuilding with document- and chunk-level content hashes, stable chunk identifiers, selective insertion, update, and deletion, and versioned manifests. A transactional index-publication strategy will be evaluated to prevent incomplete states during updates. Second, we will implement a deterministic safety layer that classifies domain scope; identifies dosage, emergency, and contraindication requests; requires adequate retrieved evidence for high-risk claims; and returns a fixed clinical-escalation response when the request cannot safely be handled. Third, we will evaluate hierarchical chunking, pre-retrieval query rewriting, and retrieval re-ranking (including cross-encoders) to increase context density and improve evidence selection under long-context conditions. Fourth, we will integrate GraphRAG topologies tailored to pediatric pain knowledge, explicitly modeling entities and relations such as pain scales, age strata, symptom patterns, pharmacological and non-pharmacological interventions, contraindications, and protocol dependencies; this is expected to improve multi-hop reasoning and source-grounded navigation across guidelines. Finally, we will study governed integration paths for temporal clinical data (including potential EHR streams), with explicit de-identification, access-control, and validation protocols before any production use.

Acknowledgments

This work has been developed within the framework of the HUPEDCARE project (Project No. 101177475, Call: ERASMUS-EDU-2024-CBHE). The project is a transnational cooperation initiative involving 15 universities across Europe, America, and Africa, coordinated by

the University of Castilla-La Mancha. The authors gratefully acknowledge the HUPEDCARE consortium for its support and collaboration in the development of this work.

Funding

This project has received funding from the European Union's Erasmus+ Programme under the action Capacity Building in the field of Higher Education (Project No. 101177475, Call: ERASMUS-EDU-2024-CBHE).

Disclaimer

The views and opinions expressed in this publication are those of the authors and do not necessarily reflect the official policy or position of the European Union. The European Commission is not responsible for any use that may be made of the information contained therein.

Declaration of Generative AI in Scientific Writing

During the preparation of this work, the authors used an AI assistant to improve language, grammar, and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the final content of the publication.

Data and Code Availability

The source code, ingestion pipeline, and implementation details for the HUPEDCARE Assistant are available for peer review and reproducibility at: https://github.com/vicegd/chatbot_hupedcare